

ЦОИ — ЛР4 — Вейвлет-обработка изображений — Вариант 5

Img_05_anls.jpg — для анализа вейвлет-декомпозиции Img_05_nois.jpg — для исследования шумоподавления (добавим гауссов шум) Img_05_count.jpg — для выделения контуров вейвлет-методом

```
clear; close all; clc;
```

Пути

```
labDir = fileparts(mfilename("fullpath"));  
imgDir = fullfile(labDir, "Рисунки");  
outDir = fullfile(labDir, "Результаты");  
if ~exist(outDir, "dir"); mkdir(outDir); end
```

Утилита: чтение изображения (в полутонах и double [0..1])

```
readGray01 = @(p) normalize01(toGrayDouble(imread(p))); % handle на 1 строке
```

1) ВЕЙВЛЕТ-ДЕКОМПОЗИЦИЯ

- сделать wavedec2() до уровня n - показать аппроксимирующие коэффициенты n-го уровня - показать детализирующие коэффициенты уровней 1..n - повторить для n=2..5 - для одного уровня сравнить разные wname (haar/db8/sym8 и др.)

```
imgAnlsPath = fullfile(imgDir, "Img_05_anls.jpg"); % изображение для анализа  
A0 = readGray01(imgAnlsPath); % исходное изображение (0..1)  
  
fig = figure("Color","w");  
imshow(A0, []); title("Var 05: Img_05_anls (исходное)");  
exportgraphics(fig, fullfile(outDir, "01_anls_original.png"), "Resolution", 200);
```



```
close(fig);

levels = 2:5; % уровни декомпозиции
wlist = ["haar","db8","sym8"]; % набор вейвлетов для сравнения

% Для каждого уровня n делаем декомпозицию (для одного wname, например sym8)
wMain = "sym8"; % базовый вейвлет для серии n=2..5
for n = levels
    [C,S] = wavedec2(A0, n, wMain); % многоуровневое 2D разложение

    % Аппроксимирующие коэффициенты на уровне n (матрица cA_n)
    cA = appcoef2(C, S, wMain, n); % аппроксимация уровня n
    saveImage(cA, fullfile(outDir, sprintf("02_anls_cA_L%d_%s.png", n, wMain)), ...
        sprintf("Аппроксимация cA (уровень %d), w=%s", n, wMain));

    % Детализирующие коэффициенты уровней 1..n (cH,cV,cD для каждого уровня)
    for j = 1:n
        cH = detcoef2("h", C, S, j); % горизонтальные детали (ур. j)
        cV = detcoef2("v", C, S, j); % вертикальные детали (ур. j)
        cD = detcoef2("d", C, S, j); % диагональные детали (ур. j)
```

```

        saveImage(cH, fullfile(outDir, sprintf("03_anls_cH_L%d_%s.png", j, wMain)), ...
            sprintf("Детали cH (уровень %d), w=%s", j, wMain));
        saveImage(cV, fullfile(outDir, sprintf("04_anls_cV_L%d_%s.png", j, wMain)), ...
            sprintf("Детали cV (уровень %d), w=%s", j, wMain));
        saveImage(cD, fullfile(outDir, sprintf("05_anls_cD_L%d_%s.png", j, wMain)), ...
            sprintf("Детали cD (уровень %d), w=%s", j, wMain));
    end
end

% Сравнение разных вейвлетов на одном выбранном уровне (например n=4)
nCmp = 4;
for wi = 1:numel(wlist)
    w = wlist(wi);
    [C,S] = wavedec2(A0, nCmp, w);
    cA = appcoef2(C, S, w, nCmp);
    saveImage(cA, fullfile(outDir, sprintf("05_compare_cA_L%d_%s.png", nCmp, w)), ...
        sprintf("Сравнение: cA уровня %d, w=%s", nCmp, w));
end

```

2) ШУМОПОДАВЛЕНИЕ (wdenctr)

- взять Img_05_nois.jpg (как исходное для эксперимента) - добавить гауссовский шум (СКО ~ 10..20% от max яркости) - подавить шум на основе wavedec2 + wdenctr - варьировать wname, уровень разложения, параметры wdenctr (thr, sorh, keepapp) - сделать таблицу метрик и выбрать лучший результат

```

imgNoisPath = fullfile(imgDir, "Img_05_nois.jpg");
X = readGray01(imgNoisPath); % базовое изображение (0..1)

sigma = 0.15; % 15% от max яркости (в диапазоне 10..20%)
Xn = X + sigma * randn(size(X)); % добавляем гауссов шум
Xn = min(max(Xn, 0), 1); % обрезка в [0..1]

saveImage(X, fullfile(outDir, "10_nois_base.png"), "Img_05_nois (база)");
saveImage(Xn, fullfile(outDir, "11_nois_added_noise.png"), ...
    sprintf("После добавления шума (sigma=%.2f)", sigma));

% Параметры перебора для wdenctr
waveList = ["haar", "db8", "sym8"];
levList = 2:5;
thrList = 0.1:0.1:0.9; % пороги (в [0..1])
sorhList = ["h", "s"]; % hard / soft
keepList = [0 1]; % КЕЕРАП (0/1)

rows = []; % сюда соберём результаты (как struct array)
k = 0;
bestMSE = inf;
best = struct();
bestImg = [];

for w = waveList

```

```

for n = levList
    [c,l] = wavedec2(Xn, n, w); % разложение зашумленного
    for thr = thrList
        for sorh = sorhList
            for keepapp = keepList
                k = k + 1;
                try
                    % wdencomp: 'gbl' – глобальный порог
                    [Xd, ~, ~, ~, ~] = wdencomp("gbl", c, l, w, n, thr, sorh, keepapp);
                    Xd = min(max(Xd, 0), 1);
                    mse = mean((X(:) - Xd(:)).^2);
                    ps = psnr(Xd, X);
                    ss = ssim(Xd, X);
                    ok = true;
                catch
                    Xd = nan(size(X));
                    mse = nan;
                    ps = nan;
                    ss = nan;
                    ok = false;
                end

                rows(k).wave = w;
                rows(k).lev = n;
                rows(k).thr = thr;
                rows(k).sorh = sorh;
                rows(k).keepapp = keepapp;
                rows(k).mse = mse;
                rows(k).psnr = ps;
                rows(k).ssim = ss;
                rows(k).ok = ok;

                if ok && mse < bestMSE
                    bestMSE = mse;
                    best.wave = w;
                    best.lev = n;
                    best.thr = thr;
                    best.sorh = sorh;
                    best.keepapp = keepapp;
                    best.mse = mse;
                    best.psnr = ps;
                    best.ssim = ss;
                    bestImg = Xd;
                end
            end
        end
    end
end
end
end
end

```

```

T = struct2table(rows);
T = sortrows(T, "mse", "ascend", "MissingPlacement", "last");
writetable(T, fullfile(outDir, "12_wdencmp_table.csv"));
save(fullfile(outDir, "12_wdencmp_table.mat"), "T", "best");

% Сохранение лучшего результата
if ~isempty(bestImg)
    saveImage(bestImg, fullfile(outDir, "13_wdencmp_best.png"), ...
        sprintf("Лучшее wdencmp: w=%s, lev=%d, thr=%.1f, %s, keep=%d, MSE=%.4g", ...
            best.wave, best.lev, best.thr, best.sorh, best.keepapp, best.mse));
end

```

3) ВЫДЕЛЕНИЕ КОНТУРОВ (вейвлет-метод)

- делаем wavedec2 - обнуляем аппроксимирующие коэффициенты (сA_N) - делаем waverec2 - усиливаем/порогируем и получаем карту контуров - подбираем уровень разложения и тип вейвлета

```

imgCountPath = fullfile(imgDir, "Img_05_count.jpg");
Y = readGray01(imgCountPath);
saveImage(Y, fullfile(outDir, "20_count_original.png"), "Img_05_count (исходное)");

% Для сравнения сохраним Sobel (как эталонный "классический" метод)
BWsob = edge(Y, "sobel");
saveBW(BWsob, fullfile(outDir, "21_count_sobel.png"), "Контурь (Sobel)");

edgeWaveList = ["haar", "db8", "sym8"];
edgeLevList = 2:5;

bestScore = -inf;
bestEdge = false(size(Y));
bestInfo = struct();

for w = edgeWaveList
    for n = edgeLevList
        [c,s] = wavedec2(Y, n, w);
        cn = c;
        nca = s(1,1) * s(1,2); % размер аппроксимации верхнего уровня
        cn(1:nca) = 0; % обнулить аппроксимацию -> оставить "детали"

        Z = waverec2(cn, s, w); % реконструкция "по деталям"
        Z = imadjust(mat2gray(Z)); % растяжение контраста для видимости

        % Порог (подбирается грубо автоматически)
        th = graythresh(Z); % порог Оцу
        BW = Z > max(0.05, 0.7*th); % чуть жёстче, чтобы меньше мусора

        % Чистка: убрать одиночные точки и очень мелкие компоненты
        BW = bwareaopen(BW, 30);
        BW = bwmorph(BW, "thin", 1);
    end
end

```

```

% Простая “оценка качества” без эталона:
% хотим, чтобы контур был заметным, но не заливал всё изображение.
density = nnz(BW) / numel(BW); % доля пикселей контура
cc = bwconncomp(BW);
nComp = cc.NumObjects;

score = -abs(density - 0.05) - 0.002*nComp; % пикселей ~5% и меньше “мусора”

% Сохраним несколько результатов на диск
saveBW(BW, fullfile(outDir, sprintf("22_count_wavelet_%s_L%d.png", w, n)), ...
    sprintf("Контур (вейвлет): w=%s, L=%d, dens=%.3f", w, n, density));

if score > bestScore
    bestScore = score;
    bestEdge = BW;
    bestInfo.wave = w;
    bestInfo.lev = n;
    bestInfo.density = density;
    bestInfo.nComp = nComp;
end
end
end

% Сохраним лучший (по метрике) вариант отдельно
saveBW(bestEdge, fullfile(outDir, "23_count_wavelet_best.png"), ...
    sprintf("Лучший (авто): w=%s, L=%d, dens=%.3f, comp=%d", ...
        bestInfo.wave, bestInfo.lev, bestInfo.density, bestInfo.nComp));

save(fullfile(outDir, "24_count_best_info.mat"), "bestInfo");

disp("Результаты в папке:");

```

Результаты в папке:

```
disp(outDir);
```

C:\Users\artem\AppData\Local\Temp\Editor_ruzou\Результаты

ЛОКАЛЬНЫЕ ФУНКЦИИ

```

function G = toGrayDouble(A)
% Перевод изображения в полутоновое double
if ndims(A) == 3
    A = rgb2gray(A);
end
G = im2double(A);
end

function X = normalize01(X)
% Нормировка в [0..1], если диапазон “поплыл”
mx = max(X(:));

```

```

if mx > 0
    X = X / mx;
end
X = min(max(X, 0), 1);
end

function saveImage(M, pathOut, ttl)
% Сохранение матрицы как картинки (imshow + exportgraphics)
f = figure("Color","w", "Visible","off");
imshow(M, []);
title(ttl, "Interpreter","none");
exportgraphics(f, pathOut, "Resolution", 200);
close(f);
end

function saveBW(BW, pathOut, ttl)
% Сохранение логической карты (контуров)
f = figure("Color","w", "Visible","off");
imshow(BW);
title(ttl, "Interpreter","none");
exportgraphics(f, pathOut, "Resolution", 200);
close(f);
end

```